CHAPTER 6

INTRODUCTION TO PROGRAMMING

CHAPTER ESSENCE AT A GLANCE:
Structured programming constructs clear computational pipelines by coupling variable declarations and memory assignment with cascaded stream input-output operations.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Standard Stream Architecture:** The standard library handles input and output as continuous sequential byte streams, coupling the keyboard to standard input stream cin and the screen to standard output stream cout.

• **Stream Cascading and Directional Operators:** The extraction operator >> pulls tokens sequentially from input streams into variables, while the insertion operator << streams values and expressions out to the display.

• **Variable Lifespan and Scope Boundaries:** Variables possess distinct lifetimes based on storage classes; local block variables reside temporarily on the call stack, while global variables maintain permanent presence throughout execution.

• **Manipulators and Field Formatting:** Output formatting manipulators like endl and setw control line breaks, field padding widths, and column alignments without altering underlying numerical values.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Keyboard Stream} \xrightarrow{\text{cin gg}} \text{Program Memory} \xrightarrow{\text{cout ll}} \text{Visual Terminal Screen}$$

Standard Input and Output Byte Stream Pipeline

$$\text{cin} \gg v_1 \gg v_2; \quad \longleftrightarrow \quad \text{cout} \ll e_1 \ll e_2 \ll \textbf{endl};$$

Cascading Extraction and Insertion Operator Syntax

$$\text{Formatted Print} = \text{cout} \ll \textbf{setw}(w) \ll \text{val} \rightarrow \text{Right-aligns val in width } w$$

Field Width Formatting via setw Manipulator

$$\text{Variable Scope} = \text{Local (Stack Frame Lifetime)} \mid \text{Global (Entire Program Lifetime)}$$

Local versus Global Variable Scope Architecture

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Using standard cin >> stops reading at whitespace; to read an entire text line containing spaces, always use cin.getline().

• **Alert:** The formatting manipulator setw() requires the <iomanip> header, while endl is defined within <iostream>.

• **Alert:** Never use an uninitialized local variable in arithmetic calculations; local variables contain random garbage values.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. What are input and output streams in C++? Name the predefined standard stream objects. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Stream Concept: An abstraction representing a continuous sequence or flow of data bytes between the program and external physical devices. **[1 Mark]**

• Predefined Stream Objects: cin (standard input stream associated with the keyboard) and cout (standard output stream associated with the screen). **[1 Mark]**

2. Explain the term cascading of input and output operators with suitable examples. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Definition: The technique of using the stream extraction operator >> or insertion operator << multiple times consecutively in a single statement. **[1 Mark]**

• Examples: Cascading input: cin >> rollNo >> marks; Cascading output: cout << "Roll: " << rollNo << " Marks: " << marks << endl; **[1 Mark]**

3. Differentiate between the newline escape sequence `\<br/>` and the manipulator `endl`. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Character vs Manipulator: \
 is a single-byte character constant that inserts a line break; endl is a specialized formatting manipulator function. **[1 Mark]**

• Buffer Flushing: \
 simply adds a newline character to the output stream buffer; endl inserts a newline and immediately flushes the output buffer to display content without delay. **[1 Mark]**

• Header Requirement: \
 works natively within string literals without extra headers; endl is provided through the <iostream> header. **[1 Mark]**

4. Explain the use of the `setw()` manipulator with a C++ code snippet. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Purpose: setw(n) sets the minimum field display width to n character columns for the immediately following output item, right-aligning content. **[1 Mark]**

• Header File: Requires inclusion of the <iomanip> header file. **[0.5 Mark]**

• Code Demonstration: #include <iostream>\
#include <iomanip>\
using namespace std;\
int main() { cout << setw(10) << 45 << endl; return 0; } (displays 45 padded with 8 leading spaces). **[1.5 Marks]**

5. Write a C++ program to input the radius of a circle and calculate its area and circumference. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Header and Setup: #include <iostream>\
using namespace std;\
int main() { float radius, area, circumference; const float π = 3.14159; **[1 Mark]**

• Input and Calculation: cout << "Enter Radius: "; cin >> radius; area = PI * radius * radius; circumference = 2 * PI * radius; **[1 Mark]**

• Output: cout << "Area: " << area << " Circumference: " << circumference << endl; return 0; } **[1 Mark]**

6. Distinguish between local variables and global variables in terms of scope, lifetime, and storage location. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Scope Boundaries: Local variables are accessible solely within the block or function where they are declared; global variables are accessible by all functions across the entire program. **[1 Mark]**

• Lifetime and Storage: Local variables reside on the call stack and are destroyed when the block exits; global variables reside in the data segment and exist for the full execution lifespan. **[1 Mark]**

• Default Initialization: Uninitialized local variables contain unpredictable garbage values; uninitialized global variables are automatically initialized to zero. **[1 Mark]**

7. Write a complete C++ program to input marks of a student in three subjects, calculate total and average marks, and print a formatted mark card. [Score: 4 Marks]

Answer & Key Valuation Steps:

• Variable Setup: #include <iostream>\
#include <iomanip>\
using namespace std;\
int main() { float mark1, mark2, mark3, total, average; **[1 Mark]**

• Data Input: cout << "Enter marks in three subjects: "; cin >> mark1 >> mark2 >> mark3; **[1 Mark]**

• Computation: total = mark1 + mark2 + mark3; average = $\frac{total}{3.0}$; **[1 Mark]**

• Formatted Print: cout << "\
--- STUDENT MARK SHEET ---" << endl; cout << "Total Marks: " << setw(8) << total << endl; cout << "Average: " << setw(8) << average << endl; return 0; } **[1 Mark]**